



Data Scientist

Technical test

1 Objetivo

El objetivo principal de esta evaluación técnica es brindar al candidato la oportunidad de demostrar sus conocimientos funcionales y técnicos relacionados con el puesto de Data Scientist. En este contexto, se entiende que alguien en este rol debe ser capaz de manejar conceptos relacionados con el desarrollo de soluciones de ciencia de datos, su ciclo de vida y todos los esfuerzos de ingeniería necesarios para poner en producción una solución de análisis avanzado.

Además de las habilidades tecnológicas, es crucial que el candidato demuestre su capacidad para extraer hallazgos significativos de los datos y de los resultados de los modelos, y traducir estos en recomendaciones accionables para el negocio, dentro del contexto del caso de uso propuesto. Queremos ver un razonamiento sólido no solo en la elección de tecnologías y algoritmos, sino también en la comprensión y aplicación de los conceptos de negocio.

La evaluación técnica debe verse como una oportunidad para demostrar las habilidades de comunicación del candidato y su capacidad para adaptarse a nuevos contextos tecnológicos. En otras palabras, incluso si el candidato no está familiarizado con algunas de las tecnologías propuestas, se espera que haga un esfuerzo para adquirir suficiente comprensión para implementar una solución (reconociendo que podría haber deuda técnica debido a las limitaciones de tiempo) o al menos proponer una solución factible.

Sin embargo, dada la dificultad y el alcance de la evaluación, no esperamos la perfección en la ejecución y los resultados. En cambio, nuestro objetivo es comprender las habilidades básicas del candidato para el puesto, así como su capacidad para adaptarse a nuevos desafíos y tecnologías.

2 Aceptación, presentación y términos

Junto con la presentación de la evaluación técnica, se solicita al candidato que proporcione una confirmación por escrito de que ha recibido y comprendido el documento, así como su compromiso de presentar los resultados en la fecha y hora acordadas.

La presentación consistirá en una sesión en vivo durante la cual el candidato, compartiendo su pantalla a través de Google Meet, demostrará interactivamente la solución desarrollada. Esto debe incluir una revisión de los diversos componentes desarrollados, así como una demostración de la funcionalidad general de la solución. También se recomienda que el candidato prepare una presentación de diapositivas que presente de forma clara y estructural la solución propuesta, incluyendo tanto sus aspectos funcionales como sus diferentes flujos y componentes.

Cualquier material producido por el candidato durante la preparación de la evaluación técnica (como código, presentaciones, diagramas, etc.) podrá ser solicitado para su posterior revisión por parte de los evaluadores y estará sujeto al mismo nivel de confidencialidad que el resto del proceso de selección.

3 Desarrollo de la prueba técnica

3.1 Datos proporcionados

El conjunto de datos proporcionado (dataset.csv) contiene una lista de clientes (identificados por la columna Customer_Id) pertenecientes a una empresa del sector de las Telecomunicaciones. También se le proporciona data_descriptions.csv, un archivo CSV que contiene las descripciones de las columnas de dataset.csv.

3.2 Propósito de la solución

3.2.1 Propósito analítico

Esta empresa nos ha pedido ayuda sobre cómo mejorar la gestión de su cartera de clientes. Específicamente, están interesados en la aplicación de técnicas analíticas para encontrar los factores asociados con las razones que llevan a un cliente a abandonar la empresa, para implementar mejoras en sus procedimientos. Para ello, verá que el resto de las columnas del conjunto de datos contienen información sobre, por ejemplo, cuántas llamadas o mensajes SMS ha recibido el cliente.

El objetivo de los usuarios de negocio de la empresa es comprender los efectos individuales de cada variable en la rotación de clientes. Además de comprender el impacto de cada variable en el modelo (feature importance) y cómo las diferentes variables aumentan o disminuyen el riesgo de churn de la empresa al puntuar a un cliente, también buscan recomendaciones accionables derivadas del análisis de datos y modelos. Esto significa no solo identificar qué variables son importantes, sino también cómo estos conocimientos se pueden traducir en estrategias concretas para retener clientes y mejorar su experiencia general.

3.2.2 Propósito funcional

El propósito funcional de la solución propuesta es construir un DAG (Grafo Acíclico Dirigido) basado en el conjunto de datos proporcionado que represente el ciclo de vida típico de construcción y aplicación de un modelo de análisis avanzado; específicamente, este DAG debe incluir los siguientes pasos:

1. Un paso de preparación de datos que cargue el tablero de entrada para el modelo de análisis avanzado de su elección.
2. Un paso para entrenar el modelo y guardar el modelo como un archivo pickle.
3. Un paso final que cargue el modelo previamente entrenado y guardado y evalúe el modelo utilizando un conjunto de datos de exclusión utilizando la métrica que considere más apropiada y muestre esas métricas de alguna manera (por ejemplo, escribiéndolas en el registro de ejecución).

3.3 Componentes y tecnologías involucradas

Para el desarrollo de esta evaluación técnica, se propone el uso de las siguientes tecnologías:

- **Motor de orquestación:** Dado que la solución requiere la programación y automatización de un flujo de trabajo de reentrenamiento y reimplementación de modelos, debe haber un componente capaz de realizar esta tarea sin interacción humana una vez que los procesos apropiados se implementen en él. Animamos a usar **Apache Airflow** si la implementación se realiza localmente.
- **Contenerización de aplicaciones:** Se recomienda **Docker** como herramienta de contenerización para garantizar un entorno de desarrollo consistente y reproducible en el que implementar Apache Airflow. Esto facilita la portabilidad del flujo de trabajo y minimiza los problemas de configuración del entorno. Docker generalmente funciona mejor en entornos Linux. Si está en Windows, se recomienda usar WSL2.

Además, todo el desarrollo debe realizarse en Python. También se recomienda gestionar el código utilizando un sistema de control de versiones.

Consulte la sección 5.1 Configuración del entorno de trabajo para obtener más detalles.

3.4 Implementación de la solución

Habiendo comprendido el propósito de la evaluación técnica y los diversos componentes y tecnologías que la solución debe incorporar, se propone el siguiente flujo de trabajo:

- **Experimentación:** Análisis de los datos de la cartera de clientes:
 - Utilizando el conjunto de datos proporcionado, el candidato debe realizar un análisis del impacto de las diferentes características del *churn* de clientes. En esta etapa, la creación de visualizaciones que permitan una fácil interpretación de los resultados será muy valorada.
 - Basándose en los conocimientos adquiridos, el candidato debe desarrollar un modelo predictivo utilizando cualquier enfoque algorítmico, junto con los paquetes o bibliotecas necesarios, para lograr los objetivos.
- **Infraestructura:** El candidato debe implementar localmente las tecnologías mencionadas en la sección 3.3, Componentes y Tecnologías Involucradas, para desarrollar el caso de uso.

3.5 Resultados esperados

Una vez finalizados los desarrollos descritos anteriormente, independientemente de las elecciones tecnológicas realizadas por el candidato, se espera lo siguiente:

- El candidato debe extraer conclusiones de negocio de los resultados del modelo y del análisis descriptivo de las variables, imaginando una posible sesión con varias partes interesadas no técnicas, y ser capaz de hacer recomendaciones sobre cómo retener clientes basándose en el análisis y los resultados del modelado.
- El candidato debe ser capaz de presentar el proceso y la metodología utilizados, justificando las decisiones tomadas con respecto al preprocesamiento de variables, el ajuste de parámetros clave, el modelado, la selección de métricas y la metodología de evaluación y validación.
- El candidato debe ser capaz de presentar, explicar y defender el uso de las tecnologías empleadas para implementar la solución propuesta, así como la estructura y modularización del código desarrollado.
- El candidato debe demostrar que es posible ejecutar las canalizaciones de procesamiento y aplicación del modelo.
- Si se ha implementado o habilitado un componente de base de datos, el candidato debe explicar y demostrar cómo se han puesto a disposición los datos.

Si alguno de estos objetivos no se cumple, se valorará positivamente, reconociendo la defensa de los enfoques adoptados y la propuesta de estrategias alternativas que habrían explorado con más tiempo.

Independientemente de lo lejos que haya podido llegar el candidato, también valoraremos lo que haya investigado, su presentación del problema y todo lo que se le ocurra para contribuir a la prueba más allá de lo que proponemos (por ejemplo, si es fuerte en visualización de datos y cree que hay algo que puede integrar, no dude en hacerlo).

3.6 Posibles extensiones

La prueba técnica es bastante libre de interpretación, por lo que se deja que se piense por un mismo, no obstante recomendamos realizar las siguientes extensiones:

- Le animamos a enriquecer la solución proporcionada y, si lo desea, a investigar cómo implementar una base de datos (PostgreSQL, por ejemplo) que le permita almacenar todos los datos del proceso (inicio, intermedios y finales) en un sistema diferente al almacenamiento local de Apache Airflow o al registro de ejecución. Esto implicará modificar el proceso para conectarse a la base de datos para lectura/escritura.
- También podría implementar una instancia de MLflow (<https://mlflow.org/>) que le permita registrar los parámetros y métricas del modelo generado, lo cual es bastante común al aplicar una metodología MLOps.
- Cualquier integración con herramientas de monitoreo que desee considerar (Prometheus y Grafana, por ejemplo) también será valorada positivamente.
- Finalmente, cualquier cosa que se le ocurra para enriquecer la solución en términos de ingeniería, ya sea integrando nuevas partes en la arquitectura u optimizando procesos tanto en términos de rendimiento como de calidad de código, será igualmente apreciada. Como se mencionó anteriormente, el candidato puede optar por abordar uno, varios o ninguno de estos puntos, dependiendo de las habilidades que desee demostrar, o incluso puede proponer otras extensiones.

4 Contacto

Si durante la preparación de la evaluación técnica el candidato tiene alguna pregunta, ya sea sobre el enfoque, aspectos técnicos o el proceso de selección, puede ponerse en contacto con los siguientes: Jesús Vicente García Hernández (jesusvicente.garcia@sdggroup.com — +34 669727656), Head of the Data Science area at SDG Group., (andres.padrones@sdggroup.com — +34 606444271), Head of the Business Science team., Mario Alberich (mario.alberich@sdggroup.com) o Joan Marchan (joan.marchan@sdggroup.com — +34 669285434), Heads of the ML Engineering team.

Además, si el candidato necesita reprogramar la fecha u hora acordada para la presentación de la evaluación técnica debido a razones personales o profesionales, puede ponerse en contacto con el equipo para encontrar una alternativa que funcione para todas las partes involucradas.

5 Apéndice

5.1 Configuración del entorno de trabajo

1. El primer punto que deberá abordar en la prueba técnica es el uso de Docker (<https://www.docker.com/>) para desplegar Apache Airflow (<https://airflow.apache.org/>). Docker es un proyecto de código abierto que automatiza el despliegue de aplicaciones dentro de contenedores de software, proporcionando una capa adicional de abstracción y automatización de la virtualización de aplicaciones en múltiples sistemas operativos. Este punto es importante porque le ayudará a configurar el entorno que proponemos a continuación.

HOW-TO: Docker generalmente se maneja mejor en entornos Linux, pero es perfectamente viable usarlo en Windows siempre que tenga Windows 10. Si está en Windows, le recomendamos usar WLS2. Puede encontrar más información sobre cómo instalar Docker en el siguiente enlace: <https://www.docker.com/get-started>

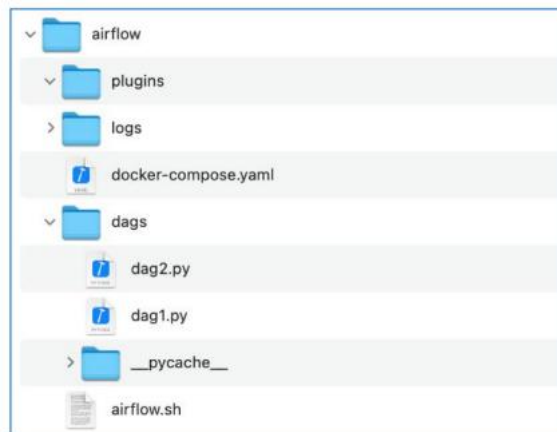
2. Una vez que esté familiarizado con Docker, debe usarlo para desplegar **Apache Airflow**. Apache Airflow es un orquestador de procesos que, a diferencia de otros, se basa en la especificación de flujos de ejecución a través de scripts de Python. El **orquestador** es una parte importante de cualquier plataforma de análisis avanzado, ya que es la pieza que le permite integrar servicios de diversas fuentes de datos y proporcionar información de forma síncrona o asíncrona a otros servicios que conectan fuentes de datos con destinos mediante la definición de tareas o actividades.

En el caso de Airflow, la orquestación entre tareas se realiza definiendo lo que se conoce como un DAG (Grafo Acíclico Dirigido). Un DAG es una colección de tareas organizadas a través de dependencias y estableciendo relaciones que definen el orden en que estas tareas deben ejecutarse (<https://airflow.apache.org/docs/apache-airflow/stable/concepts/dags.html>).

3. Para facilitar la comprensión de cómo usar Apache Airflow desplegado en un contenedor Docker, le proporcionamos las siguientes sugerencias:

HOW-TO: Siga las instrucciones del siguiente enlace para desplegar Apache Airflow en Docker usando un archivo de configuración ".yaml" llamado Docker Compose: <https://airflow.apache.org/docs/apache-airflow/stable/start/docker.html>.

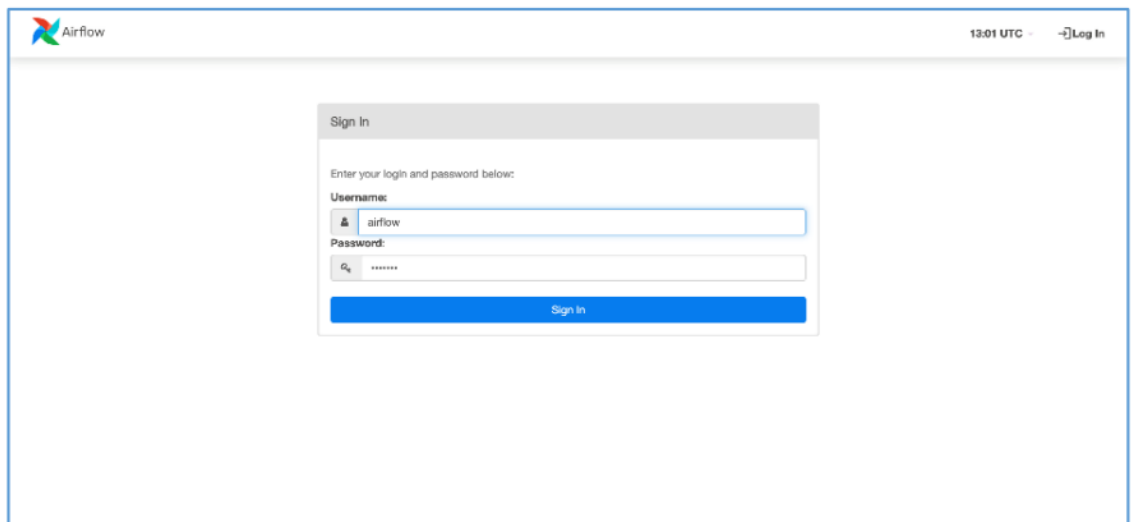
4. Básicamente, los pasos que tendrá que seguir son los siguientes:
 - [Descargue el archivo de configuración](#) y colóquelo en una nueva carpeta de proyecto junto con otras tres subcarpetas (\dags, \logs y \plugins). La carpeta \dags es donde tendrá que guardar los flujos de tareas que implementará posteriormente para resolver esta prueba técnica. La carpeta del proyecto tendrá entonces este formato inicial:



- Como segundo paso, tendrá que **inicializar la base de datos** que soporta los datos manejados por el orquestador usando el comando **docker-compose up airflow-init**. Este comando se ejecuta a través de la Terminal en la carpeta donde se ha almacenado el docker-compose.yaml y es responsable de inicializar todos los servicios necesarios para levantar Airflow en Docker. Así:

```
angel@MacBook-Pro-de-Angel airflow % docker-compose up airflow-init
Creating network "airflow_default" with the default driver
Creating airflow_redis_1 ... done
Creating airflow_postgres_1 ... done
Creating airflow_airflow-init_1 ... done
Attaching to airflow_airflow-init_1
airflow-init_1 | The container is run as root user. For security, consider using a regular user account.
airflow-init_1 | /home/airflow/.local/lib/python3.7/site-packages/sqlalchemy/dialects/postgresql/base.py:357
3 SAWarning: Predicate of partial index idx_dag_run_queued_dags ignored during reflection
airflow-init_1 | /home/airflow/.local/lib/python3.7/site-packages/sqlalchemy/dialects/postgresql/base.py:357
3 SAWarning: Predicate of partial index idx_dag_run_running_dags ignored during reflection
airflow-init_1 | DB: postgresql+psycopg2://airflow:***@postgres/airflow
airflow-init_1 | [2022-04-06 13:16:16,415] {db.py:919} INFO - Creating tables
airflow-init_1 | INFO [alembic.runtime.migration] Context impl PostgresqlImpl.
airflow-init_1 | INFO [alembic.runtime.migration] Will assume transactional DDL.
airflow-init_1 | Upgrades done
airflow-init_1 | [2022-04-06 13:16:27,578] {manager.py:512} WARNING - Refused to delete permission view, ass
oc with role exists DAG Runs.can_create User
airflow-init_1 | airflow already exist in the db
airflow-init_1 | 2.2.5
airflow-airflow-init_1 exited with code 0
angel@MacBook-Pro-de-Angel airflow %
```

- Finalmente, tendrá que **dejar que Airflow se ejecute** usando el comando **docker-compose up**. Este comando hará que Airflow sea accesible desde un navegador a través del enlace <http://localhost:8080/>. Las instrucciones proporcionadas crearán un usuario con permisos de administrador para iniciar sesión en la interfaz web de Airflow con el usuario airflow y la contraseña airflow. Sabrá que ha funcionado si ve una pantalla de inicio de sesión como esta cuando inicie sesión en su navegador a través de la dirección localhost:



- Una vez iniciada la sesión, verá una lista completa de DAGs de ejemplo como la siguiente:

DAG	Owner	Runs	Schedule	Last Run	Next Run	Recent Tasks
example_bash_operator	airflow	0/0	@daily	2022-04-05, 00:00:00		
example_branch_datetime_operator_2	airflow	0/0	@daily	2022-04-05, 00:00:00		
example_branch_dop_operator_v3	airflow	0/0	@daily	2022-04-06, 12:57:00		
example_branch_labels	airflow	0/0	@daily	2022-04-05, 00:00:00		
example_branch_operator	airflow	0/0	@daily	2022-04-05, 00:00:00		
example_complex	airflow	0/0	None			
example_dag_decorator	airflow	0/0	None			
example_external_task_marker_child	airflow	0/0	None			

- Además, verá los DAGs que ha dejado en la carpeta dag del proyecto. Por ejemplo, en nuestro caso hemos dejado dos dags llamados "tutorial_1" y "tutorial_2".

DAG	Owner	Runs	Schedule	Last Run	Next Run	Recent Tasks	Action
tutorial	airflow	1/0	1 day, 0:00:00	2022-04-06, 11:00:47	2022-04-06, 11:00:45		
tutorial_1	airflow	0/0	1 day, 0:00:00		2022-04-05, 12:55:23		
tutorial_2	airflow	0/0	1 day, 0:00:00		2022-04-05, 12:55:23		
tutorial_etl_dag	airflow	0/0	None				
tutorial_taskflow_api_etl	airflow	0/0	None				
tutorial_taskflow_api_etl_virtualenv	airflow	0/0	None				

Es muy común quedarse atascado durante el despliegue de Airflow en Docker, pero para evitar estos problemas es muy importante seguir todas las instrucciones provistas y familiarizarse con la interfaz de Airflow y los DAGs



SDG Group

✉ <https://www.sdgggroup.com/>

📧 info.es@sdgggroup.com